Neural Machine Translation

[1] Saicharan Polishetty, [2] Nikhil Bharadwaj, [3] Nishanth M
[1, 2, 3,4] Department of Computer Science and Engineering, PES University
[1]saicharan19022002@gmail.com, [2]nikhil.yellapragada@gmail.com, [3]duos16112001@gmail.com

***Abstract*— India is a multilingual country; different states have different territorial languages but not all Indians are polyglots. There are 18 constitutional languages and ten prominent scripts. The majority of the Indians, especially the remote villagers, do not understand, read or write English, therefore implementing an efficient language translator is needed. Machine translation systems, that translate text form one language to another, will enhance the knowledgeable society of Indians without any language barrier. English, being a universal language and Hindi, the language used by the majority of Indians, we propose an English to Hindi machine translation system design based on declension rules. This paper also describes the different approaches of Machine Translation**

Keywords- English to Hindi translation; Hybrid Based Machine Translation; Rule based; Statistical Machine Translation.

INTRODUCTION

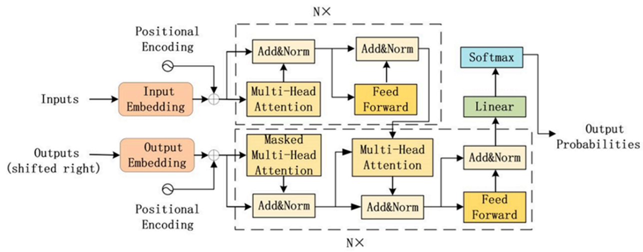
Machine Translation (MT) Systems are used for automated translation of one natural language to another. Machine translation is the research field of Natural Language Processing(NLP)which aims to fill the gap communication among the different sections of societies.
Human translation of any language is time consuming and expensive. By the use of machine translation systems, we can reduce the time and cost of human translators. Hindi, the official language of India, is used by about 400 million people. The motivation behind the project is the English to Hindi machine translation system is that many documents and records like government records, historical, news etc. are written in English, which is not popular among the remote villagers of India. However, the rural villagers, especially in north India, apart from their native language, know Hindi. Hence, there is a need for an automatic translation from English to Hindi.

I. LITERATURE SURVEY

[1]The paper proposes a transformer network to be used instead of RNN for machine translation due to following reasons :-

It can only process word-by-word and result in slow training speed; when the sentence is too long, gradient disappearance and explosion reduce the accuracy of translation.

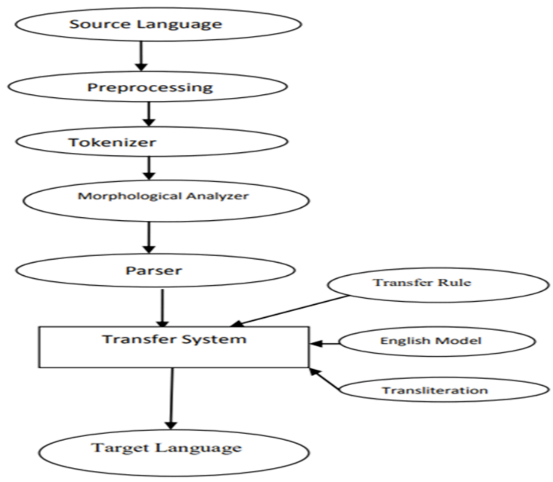
The proposed architecture for the transformer for machine translation , is composed of is the use of sinusoidal positional encoding, a mask(used make the value of mask in position of pad equal 1),look-ahead mask (make the value of mask in position of future token equal 1),multi-head attention, encoder, decoder, transformer, configuration.



The proposed model achieved 80% accuracy and the model had been tested in the actual translation and had achieved a better effect. The model can be preliminarily applied to the translation of languages with large lexical database

[2]In a Transfer Based System a source language is transformed into an abstract representation then its equivalent is generated for target language using bilingual dictionaries and grammar rules , it contains 3 phases - Analysis phase , transfer phase and synthesis phase .

First the source language is pre-processed where all the punctuations and special characters are taken care of , Then we send it through a tokenizer which divides the sentence into meaningful unit known as tokens



It then goes through source language morphological analyzer where the parts of speech of the source language words is identified and are parsed with the help of CYK algorithm where the translation rules and syntactic structure is verified and passed to the next phase which is the transfer system which contains the transfer rules , the english model and the transliteration part which which deals with proper nouns as the sentences can contain names also and finally the target language is generated.

II. Proposed Solution

The proposed solution consists of dealing with 2 different models of traditional machine translation models as we can compare them in order to get better insights on which model performs better so that the end - user who is building a machine translation model can use the better model to yield better results.

Encoder-Decoder architecture can handle inputs and outputs that are both variable-length sequences, thus is suitable for sequence translation problems such as Machine Translation. The Encoder circuit converts the applied information signal into a coded digital bit stream. Decoder performs reverse operation and recovers original signal from the coded bits. We use Adam optimizer for Attention mechanism.

LSTM (Long Short Term Memory) primarily solves the vanishing gradient problem . Information in LSTM can be stored, written, or read via gates that open and close. It's capable of learning order dependence in sequence prediction problems. Even LSTM was implemented using the Encoder and Decoder model without Attention mechanism but using RMSProp optimizer.

A. *Dataset Description*

The IIT Bombay English-Hindi corpus contains parallel corpus for English-Hindi as well as monolingual Hindi corpus collected from a variety of existing sources and corpora developed at the center for Indian Language Technology, IIT Bombay over the years. This page describes the corpus. This corpus has been used at the Workshop on Asian Language Translation Shared Task since 2016 the Hindi-to-English and English-to-Hindi languages pairs and as a pivot language pair for the Hindi-to-Japanese and Japanese-to-Hindi language pairs.

B. *Preprocessing*

As basic pre-processing, we need to remove punctuation, numerals, extra spaces and convert them to lowercase which is pretty standard stuff encapsulated within a function.

In addition to running the sentences, they need to be sanitized to remove the english characters they contain as well. This actually caused NaN bugs during training which consumed a lot of time in debugging. We can avert the issue by replacing any English character with an empty string.

In sequence to sequence translation every word in a sentence should contain a unique identity, representing each word in a language as a one-hot vector or giant vector containing zero except one in the whole vector.

C. *Training and Testing*

The given dataset was trained and tested using 2 different models which are the most used in-case of language translation that is LSTM and Attention mechanism for the encoder-decoder model , We have used 2 models to compare and get more insights on which is better so that any end-user who wants to work with the language translation can use the better model to get better results.

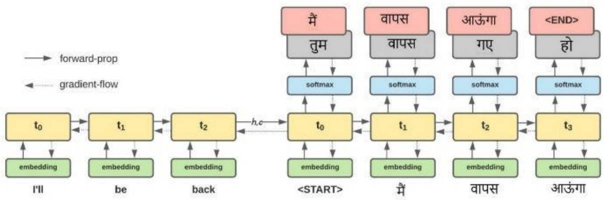
III. Implementation and results

Initially the dataset was taken from IIT-Bombay site where in the dataset contained 2 files , the english sentence and the corresponding hindi sentence which was imported and zipped into a single file for simplicity.

The preprocessing part was done where in all the required processing on the dataset was done such as removal of punctuation marks, converting all the text from upper case to lowercase etc , The size of the dataset was reduced from about 1 Million sentences to 25,000 sentences in order to favor the training time.

A) LSTM

The way the NMT model works can be understood as a two phase process. During the first phase, the encoder block ingests the input sequence one word at a time. Since, we are using an LSTM layer, with the consumption of each word, the internal state vectors will get updated. After the final word, we care only about the final values of these vectors to be fed into the decoder. The second phase then works by ingesting the encoder's final hidden state vector and also consuming each token of the target sentence, beginning with <START>. The output produced for each token will be matched with what is expected and error gradients are generated to update the model. Upon exhaustion of the target inputs, we expect our model to produce an <END> token denoting the completion of this data-sample. This can be better understood from the below diagram.



As, we can see the decoder LSTM is initialized with the final hidden state vector of the encoder. We can understand this step as a transfer of thought/summary of the input sentence to the decoder LSTM. The grey boxes indicate the output of the LSTM unit and the red boxes the target token at each time-step. When the first token <START> is fed into the decoder LSTM, we expect it to produce मैं, however it produces तुम accounting for a contribution to the error gradient (to be used for backpropagation through time).We train the model for 70 epochs with rmsprop as optimizer,loss function as SPARSE CATEGORICAL CROSS ENTROPY.Then we test the model with test split,and results obtained from the model are shown in the below figure

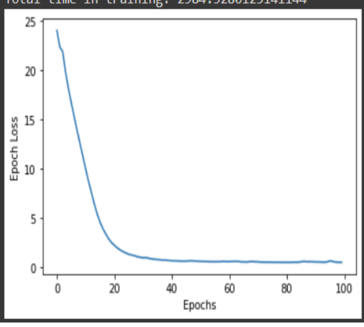
```
Input:  highlight duration
Prediction:   अवधि को अवधि
Translated Reference:  हाइलाइट अवधि
Dataset Reference:  अवधि को हाइलाइट रकें

Input:  the duration of the highlight box when selecting accessible nodes
Prediction:   पता समय की प्रतिक्रिया प्राप्त करें थी
Translated Reference:  पहुँच योग्य नोड्स को चुनते समय हाइलाइट बक्से की अवधि
Dataset Reference:  पहुंचनीय आसंधि नोड को चुनते समय हाइलाइट बक्से की अवधि

Input:  highlight border color
Prediction:   सीमांत दिखाएँ
Translated Reference:  किनारा रंग उभारें
Dataset Reference:  सीमांत बोर्डर के रंग को हाइलाइट करें

Input:  the color and opacity of the highlight border
Prediction:   हाइलाइट किए गए सीमांत का रंग और पारदर्शिता।
Translated Reference:  हाइलाइट किनारे की रंग और अपारदर्शिता
Dataset Reference:  हाइलाइट किए गए सीमांत का रंग और अपारदर्शिता।

Input:  highlight fill color
Prediction:   कार्य को रंग को बंद करें
Translated Reference:  भरने के रंग को उभारें
Dataset Reference:  भराई के रंग को हाइलाइट करें

Input:  the color and opacity of the highlight fill
Prediction:   हाइलाइट किया गया रंग का
Translated Reference:  हाइलाइट भरने की रंग और अपारदर्शिता
Dataset Reference:  हाइलाइट किया गया भराई का रंग और पारदर्शिता।

Input:  api browser
Prediction:   एपीआई विचरक
Translated Reference:  एपी ब्राउज़र
Dataset Reference:  एपीआई विचरक

Input:  browse the various methods of the current accessible
Prediction:   दो जा के साथ के नॉइसरिडक्शन
Translated Reference:  वर्तमान पहुँच योग्य की विभिन्न विधियों को ब्राउज़ करें
Dataset Reference:  इस समय जिसे प्राप्त किया गया हो उसकी विभिन्न विधियों मेथड में विचरण करें

Input:  hide private attributes
Prediction:   निजी गुणों को छिपाएं
Translated Reference:  निजी गुण छुपाएँ
Dataset Reference:  निजी गुणों को छिपाएं
```

B) SEQUENCE 2 SEQUENCE WITH ATTENTION

These sentences were tokenized into words and then we did a test and train split. After that comes the initialization of Encoder and Decoder where the sequence of tokens were passed and returns a 2-dim vector.

Decoder takes it as an input and we send the corresponding translated sentence inorder to train the model. After the training is done, we do the testing where we give the input from the test split which is being translated to the target language.

```
Epoch No.: 95 Time: 28.766494750976562
Epoch No.: 96 Time: 29.055413722991943
Epoch No.: 97 Time: 28.972052574157715
Epoch No.: 98 Time: 29.112112283706665
Epoch No.: 99 Time: 28.957338333129883
All Epoch Losses: [24.05240754823427, 22.374680338679134, 21.919303842493004, 19.82954875
Total time in training: 2984.5280125141144
```



As you can see, the error is getting reduced after every Epoch.

```
[ ] print(translate_sentence("give your application an accessibility workout"))

    sentencestart give your application an accessibility workout sentenceend
    ['अपने अनुप्रयोग को पहुंचनीयता व्यायाम का लाभ दें sentenceend']

    # print translated sentence
    print(translate_sentence("interactive console for manipulating currently selected accessible"))

    sentencestart interactive console for manipulating currently selected accessible sentenceend
    ['इस समय चुने गए एक्सेसेबेल से काम लेने के लिए अंतर्क्रियात्मक कन्सोल sentenceend']

[ ] print(translate_sentence("accerciser accessibility explorer"))

    sentencestart accerciser accessibility explorer sentenceend
    ['एक्सेर्साइसर पहुंचनीयता अन्वेषक sentenceend']
```

IV. Conclusion

In this paper, we have presented an effective technology for English to Hindi MT based on declensions. We have used the LSTM and Attention mechanism. LSTM is effectively used in translation applications by adding intermediate state information to directly propagate backwards. The LSTM model has good generalization so that the performance of the translation model is further improved.

The Attention mechanism is a very useful technique in NLP tasks as it increases the accuracy. The only disadvantage of the Attention mechanism is that it is very time consuming and hard to parallelize.

After training the models we have observed that the Attention mechanism works better than the LSTM in translation because the Attention mechanism works better for long sentences and its ability to identify information in an input. Whereas LSTM is able to predict easy words while struggling for large sentences.